```
    @isbn = isbn
    @price = Float(price)
  end

  def price=(new_price)
    @price = new_price
  end

  # ...
end

book = BookInStock.new("isbn1", 33.80)
puts "ISBN      = #{book.isbn}"
puts "Price     = #{book.price}"
book.price = book.price * 0.75        # discount price
puts "New price = #{book.price}"
```

produces:

```
ISBN      = isbn1
Price     = 33.8
New price = 25.349999999999998
```

The assignment `book.price = book.price * 0.75` invokes the method `price=` in the `book` object, passing it the discounted price as an argument. If you create a method whose name ends with an equals sign, that name can appear on the left side of an assignment (and the Ruby parser will ignore whitespace between the end of the name and the equals sign, which is how `book.price =` gets parsed to the method named `price=`. You can even treat the setter method like a regular method invocation if you want—`book.price = 1.50` is identical to the somewhat odder-looking `book.price=(1.50)`.

Again, Ruby provides a shortcut for creating these simple attribute-setting methods. If you want a write-only accessor, you can use the form `attr_writer`, but that's fairly rare. You're far more likely to want both a reader and a writer for a given attribute, so you'll use the handy-dandy `attr_accessor` method:

tut_classes/stock_stats/book_in_stock_6.rb

```
class BookInStock
  attr_reader :isbn
  attr_accessor :price

  def initialize(isbn, price)
    @isbn = isbn
    @price = Float(price)
  end
  # ...
end

book = BookInStock.new("isbn1", 33.80)
puts "ISBN      = #{book.isbn}"
puts "Price     = #{book.price}"
book.price = book.price * 0.75        # discount price
puts "New price = #{book.price}"
```

produces:

```
ISBN      = isbn1
Price     = 33.8
New price = 25.349999999999998
```